

KAN-16

Block"1" - Validar usuarios nuevos/existentes (Backend)

- Implementar lógica para verificar si el usuario autenticado ya existe en la base de datos
- Crear registro automático para nuevos usuarios
- Recuperación de información para usuarios existentes.

- - - - -

El objetivo principal de este bloque consiste en implementar la lógica del lado del servidor (Backend) para gestionar el flujo de autenticación mediante el protocolo Google OAuth 2.0, utilizando el paquete oficial `laravel/socialite`. El sistema debe cumplir estrictamente con las siguientes reglas de negocio:

1. **Validación de existencia:** Verificar si el correo devuelto por Google ya se encuentra registrado en la base de datos local.
2. **Registro automatizado:** Si el usuario es nuevo, el sistema debe darlo de alta automáticamente con un rol básico de estudiante y estado activo.
3. **Recuperación y unificación:** Si el usuario ya existe (por ejemplo, creado previamente por migración o administración manual), se debe recuperar su información y enlazar de forma transparente su identificador único de Google (`google_id`).

El desarrollo de este requerimiento se concentró **exclusivamente** en dos archivos del proyecto, garantizando un impacto aislado y limpio en la arquitectura existente:

- **`app/Models/Usuario.php` (Adaptación del Modelo):**
Se modificó la herencia base de la clase desde `Eloquent\Model` hacia `Illuminate\Foundation\Auth\User as Authenticatable`. Este cambio subsanó un conflicto de tipos en el framework, dotando al modelo de los contratos necesarios para interactuar con las fachadas nativas de autenticación y permitiendo que sea reconocido correctamente por el sistema de autenticación de Laravel.
- **`app/Http/Controllers/AuthController.php` (Lógica de Autenticación y API):**
Dentro de este controlador se concentró toda la lógica de negocio del KAN-16 mediante tres métodos clave:
 1. **`redirect()`:** Redirige al usuario a la pasarela de Google utilizando `->stateless()` para evitar conflictos de cookies en entornos locales (`localhost / 127.0.0.1`).
 2. **`callback()`:** Recibe el usuario de Google de forma *stateless*, ejecuta la validación de existencia por correo, realiza la inserción del nuevo usuario (junto con la creación de su perfil base en la tabla `perfiles` para mitigar errores de integridad) o actualiza el `google_id` si ya existía. Finalmente devuelve el JSON con el `accessToken`.
 3. **`usuario(Request $request)`:** El nuevo método adaptado para la API *stateless*. Extrae el token Bearer de la cabecera HTTP, consulta la validez del mismo directamente con los servidores de Google mediante `Socialite::driver('google')->stateless()->userFromToken($token)` y retorna los datos limpios del usuario en formato JSON.

Aunque las rutas del sistema ya estaban predefinidas en el repositorio por el equipo, la validación del flujo se completó exitosamente gracias a la lógica implementada en

`AuthController.php`. El endpoint de consulta opera bajo el enfoque de Tokens Bearer (Stateless):

- **GET /api/usuario** (Ubicado en `routes/api.php` y apuntando a `AuthController@usuario`):
Lee el token que el Frontend envía en la cabecera `Authorization: Bearer <token>`, valida su autenticidad con Google y recupera la información del usuario de la base de datos local en tiempo real sin requerir sesiones del servidor ni cookies del navegador.

La verificación y el aseguramiento de la calidad del código se realizaron mediante dos fases de pruebas en caliente:

1. **Simulación de Peticiones API (Postman / Thunder Client):** Se validó el endpoint `GET /api/usuario` inyectando manualmente el token de Google en la cabecera `Authorization: Bearer <token>`. El sistema procesó la solicitud de forma 100% stateless, conectándose con la API de Google y devolviendo de manera exitosa el JSON correspondiente al usuario autenticado.
2. **Persistencia Física de Datos (HeidiSQL):** Se inspeccionó la base de datos local bajo el entorno Laragon mediante consultas SQL directas (`SELECT * FROM usuarios;` y `SELECT * FROM perfiles;`), constatando lo siguiente:
 - Correcto registro del `google_id` y resguardo de datos sin duplicaciones.
 - Creación instantánea y enlazada del perfil del alumno asociado al `usuario_id` recién generado.